Explanatory Notes for Section 3.2 Regularization

Let us start with the metric used

$$p_{i=1..K,j=1..K} = 1 - \chi^2 \left[\frac{\sum_{n=1}^{N_j} \sqrt{(x_n - \bar{x}_i)^T C^{-1} (x_n - \bar{x}_i)}}{N_j}, 2 \right]$$

First Lets us look into the inner most product

$$\sqrt{(x_n - \bar{x}_i)^T C^{-1} (x_n - \bar{x}_i)}$$

This represents the deviation of a point (n) in cluster j from the mean of the cluster i, normalized with respect to the covariance (C) of cluster i. This metric is normally called as the Mahalanobis distance.

Lets us first look into an event where there were irregularities which we sought to address based on the metric. Look at the plots below, which shows the cluster ID as a text and each cluster is differently colored.

In [2]:
```
import os
os.getcwd()
```

Out[2]:
```
'C:\\Users\\alarokia\\Desktop\\article_figures\\figure_3'
```

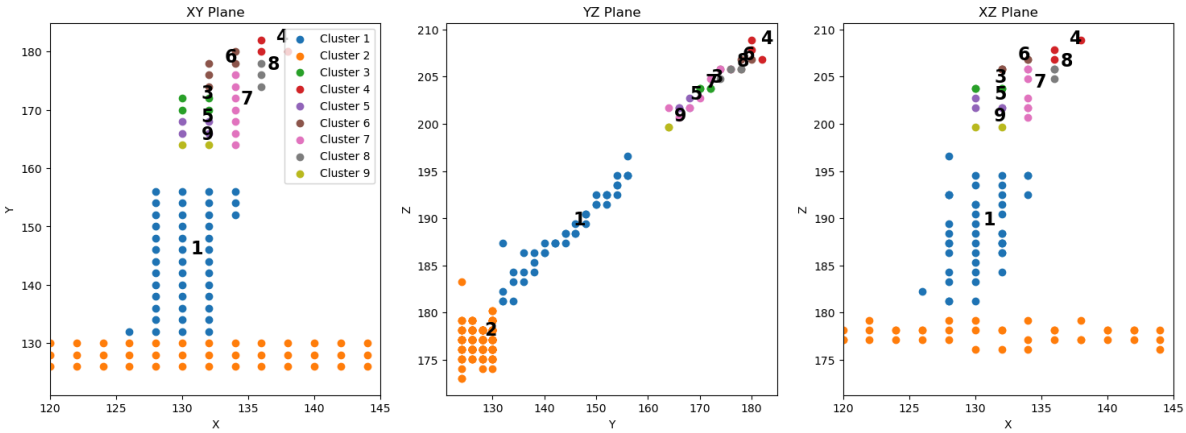
Continue reading
In the event, I remove the data points that looked like pileup, which you asked to remove

In [17]:
```
import numpy as np
import matplotlib.pyplot as plt

# Load the data from the .npy file
data = np.load('figure_3.npy')

# Extract columns for clusters, x, y, z
clusters = data[:, 0]
x = data[:, 1]
y = data[:, 2]
z = data[:, 3]

# Condition to keep points with z >= 150 and either z <= 200 or (z > 200 and y >= 1
condition = (z >= 150) & ((z <= 200) | (y >= 135))

# Apply the condition to filter the data
filtered_data = data[condition]

# Remove clusters 3 and 4
filtered_data = filtered_data[~np.isin(filtered_data[:, 0], [3, 4])]

# Reassign new cluster IDs starting from 1
unique_clusters = np.unique(filtered_data[:, 0])
for new_id, old_id in enumerate(unique_clusters, start=1):
    filtered_data[filtered_data[:, 0] == old_id, 0] = new_id

# Save the filtered data into a new file
np.save('filtered_figure_3_with_new_ids.npy', filtered_data)
```

```
# Extract columns from the filtered data for clusters, x, y, z
clusters = filtered_data[:, 0]   # cluster labels
x = filtered_data[:, 1]          # x coordinates
y = filtered_data[:, 2]          # y coordinates
z = filtered_data[:, 3]          # z coordinates

# Print the new cluster ID and the number of points in each cluster
unique_clusters = np.unique(clusters)
for cluster in unique_clusters:
    num_points = np.sum(clusters == cluster)
    print(f'Cluster {int(cluster)} has {num_points} points.')

# Define a colormap
cmap = plt.colormaps.get_cmap('tab10')

# Create the figure and subplots with increased size
fig, axes = plt.subplots(1, 3, figsize=(18, 6))  # Increased figure size

# Scatter plot for xy plane
for idx, cluster in enumerate(unique_clusters):
    mask = clusters == cluster
    axes[0].scatter(x[mask], y[mask], label=f'Cluster {int(cluster)}', color=cmap(i

    # Calculate mean of x and y for cluster
    mean_x = np.mean(x[mask])
    mean_y = np.mean(y[mask])

    # Annotate cluster ID near the mean in the xy plane with an offset
    axes[0].annotate(f'{int(cluster)}', xy=(mean_x, mean_y),
                     xytext=(5, 5), textcoords='offset points',
                     fontsize=16, fontweight='bold', color='black')

# Set xlim for xy plane
axes[0].set_xlim(120, 145)
axes[0].set_title('XY Plane')
axes[0].set_xlabel('X')
axes[0].set_ylabel('Y')

# Scatter plot for yz plane
for idx, cluster in enumerate(unique_clusters):
    mask = clusters == cluster
    axes[1].scatter(y[mask], z[mask], label=f'Cluster {int(cluster)}', color=cmap(i

    # Calculate mean of y and z for cluster
    mean_y = np.mean(y[mask])
    mean_z = np.mean(z[mask])

    # Annotate cluster ID near the mean in the yz plane with an offset
    axes[1].annotate(f'{int(cluster)}', xy=(mean_y, mean_z),
                     xytext=(5, 5), textcoords='offset points',
                     fontsize=16, fontweight='bold', color='black')

axes[1].set_title('YZ Plane')
axes[1].set_xlabel('Y')
axes[1].set_ylabel('Z')

# Scatter plot for xz plane
for idx, cluster in enumerate(unique_clusters):
    mask = clusters == cluster
    axes[2].scatter(x[mask], z[mask], label=f'Cluster {int(cluster)}', color=cmap(i

    # Calculate mean of x and z for cluster
    mean_x = np.mean(x[mask])
    mean_z = np.mean(z[mask])
```

```
    # Annotate cluster ID near the mean in the xz plane with an offset
    axes[2].annotate(f'{int(cluster)}', xy=(mean_x, mean_z),
                     xytext=(5, 5), textcoords='offset points',
                     fontsize=16, fontweight='bold', color='black')

# Set xlim for xz plane
axes[2].set_xlim(120, 145)
axes[2].set_title('XZ Plane')
axes[2].set_xlabel('X')
axes[2].set_ylabel('Z')

# Add legend to the first plot
axes[0].legend(loc='upper right')

# Show the plot with adjusted layout
# plt.tight_layout(rect=[0, 0, 1, 0.95])  # Adjust margins to avoid tight layout is
plt.show()
```

```
Cluster 1 has 43 points.
Cluster 2 has 237 points.
Cluster 3 has 4 points.
Cluster 4 has 3 points.
Cluster 5 has 4 points.
Cluster 6 has 5 points.
Cluster 7 has 7 points.
Cluster 8 has 3 points.
Cluster 9 has 2 points.
```



In [28]:
```
filtered_data
```

Out[28]:
```
array([[  1.        , 134.        , 156.        , 194.55999756],
       [  1.        , 128.        , 156.        , 196.60800171],
       [  1.        , 132.        , 156.        , 194.55999756],
       ...,
       [  8.        , 136.        , 174.        , 204.80000305],
       [  9.        , 132.        , 164.        , 199.67999268],
       [  9.        , 130.        , 164.        , 199.67999268]])
```

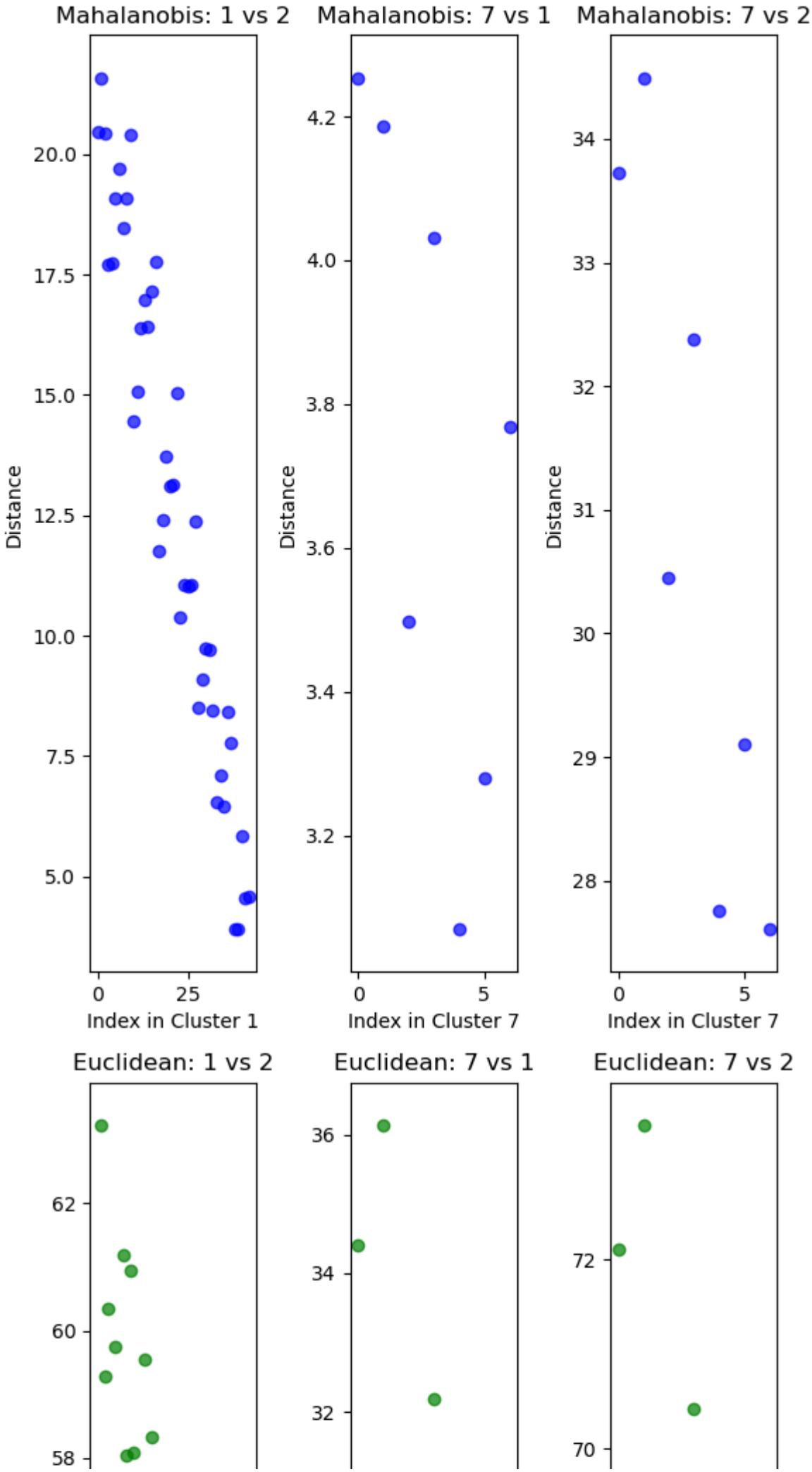
Continue Reading

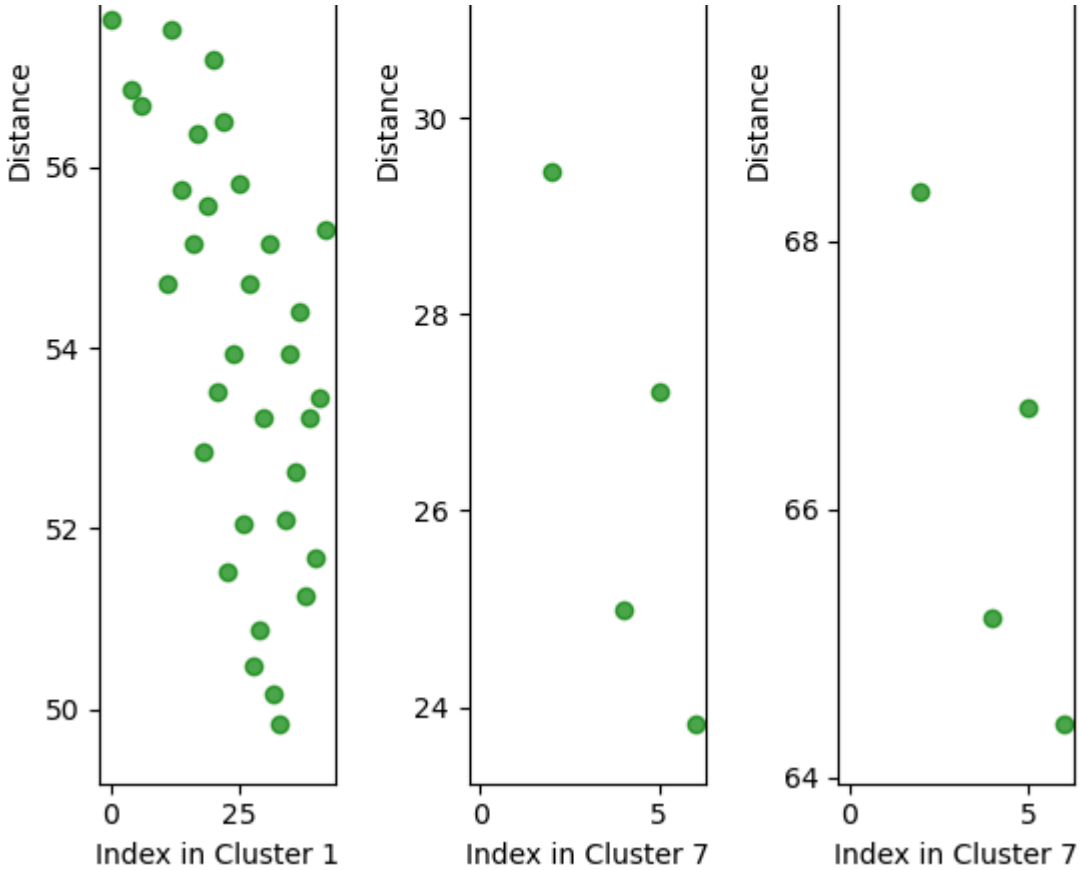
In the figure above, the cluster labels are marked as text. Now, we attempt to calculate two distances (1) Euclidean $\sqrt{(x_n - \bar{x}_i)^T (x_n - \bar{x}_i)}$ and (2) Mahalanobis $\sqrt{(x_n - \bar{x}_i)^T C^{-1} (x_n - \bar{x}_i)}$. This way we can visualize the effect of normalizing the euclidean distance by the covariance matrix.

In [21]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Load the data from the .npy file
data = filtered_data

# Extract columns for clusters, x, y, z
clusters = data[:, 0]   # cluster labels
xyz_data = data[:, 1:4]  # [x, y, z] coordinates

# Define the pairs of clusters to analyze
pairs = [(1, 2), (7, 1), (7, 2)]

# Prepare the figure for the plots
plt.figure(figsize=(15, 15))

for idx, (cluster_a, cluster_b) in enumerate(pairs, start=1):
    # Get the data points for the clusters
    cluster_a_data = xyz_data[clusters == cluster_a]
    cluster_b_data = xyz_data[clusters == cluster_b]

    # Print shapes and contents of the clusters
#     print(f'Cluster {cluster_a}: Shape = {cluster_a_data.shape}, Data = {cluster_
#     print(f'Cluster {cluster_b}: Shape = {cluster_b_data.shape}, Data = {cluster_

    # Check if cluster_b_data has enough unique points
    if cluster_b_data.shape[0] < 2:
        print(f'Skipping pair ({cluster_a}, {cluster_b}) due to insufficient data i
        continue

    # Calculate the mean vector for the reference cluster (cluster B)
    mean_b = np.mean(cluster_b_data, axis=0)

    # Calculate the covariance matrix for cluster B
    cov_matrix = np.cov(cluster_b_data.T)
    inv_cov_matrix = np.linalg.inv(cov_matrix)

    # Calculate the Mahalanobis distances for points in cluster A against the mean
    mahalanobis_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        mahalanobis_distance = np.sqrt(diff @ inv_cov_matrix @ diff.T)
        mahalanobis_distances.append(mahalanobis_distance)

    # Calculate the Euclidean distances for points in cluster A against the mean of
    euclidean_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        euclidean_distance = np.sqrt(diff @ diff.T)
        euclidean_distances.append(euclidean_distance)

    # Plotting Mahalanobis Distance
    plt.subplot(2, 8, idx)
    plt.scatter(range(len(mahalanobis_distances)), mahalanobis_distances, color='bl
    plt.title(f'Mahalanobis: {cluster_a} vs {cluster_b}')
    plt.xlabel('Index in Cluster ' + str(cluster_a))
    plt.ylabel('Distance')

    # Plotting Euclidean Distance
    plt.subplot(2, 8, idx + 8)
    plt.scatter(range(len(euclidean_distances)), euclidean_distances, color='green'
    plt.title(f'Euclidean: {cluster_a} vs {cluster_b}')
    plt.xlabel('Index in Cluster ' + str(cluster_a))
    plt.ylabel('Distance')

# Show the plot
```

```
plt.tight_layout()
plt.show()
```


Mahalanobis: 1 vs 2
Mahalanobis: 7 vs 1
Mahalanobis: 7 vs 2
Distance
Index in Cluster 1
Index in Cluster 7
Index in Cluster 7
Euclidean: 1 vs 2
Euclidean: 7 vs 1
Euclidean: 7 vs 2



Continue Reading

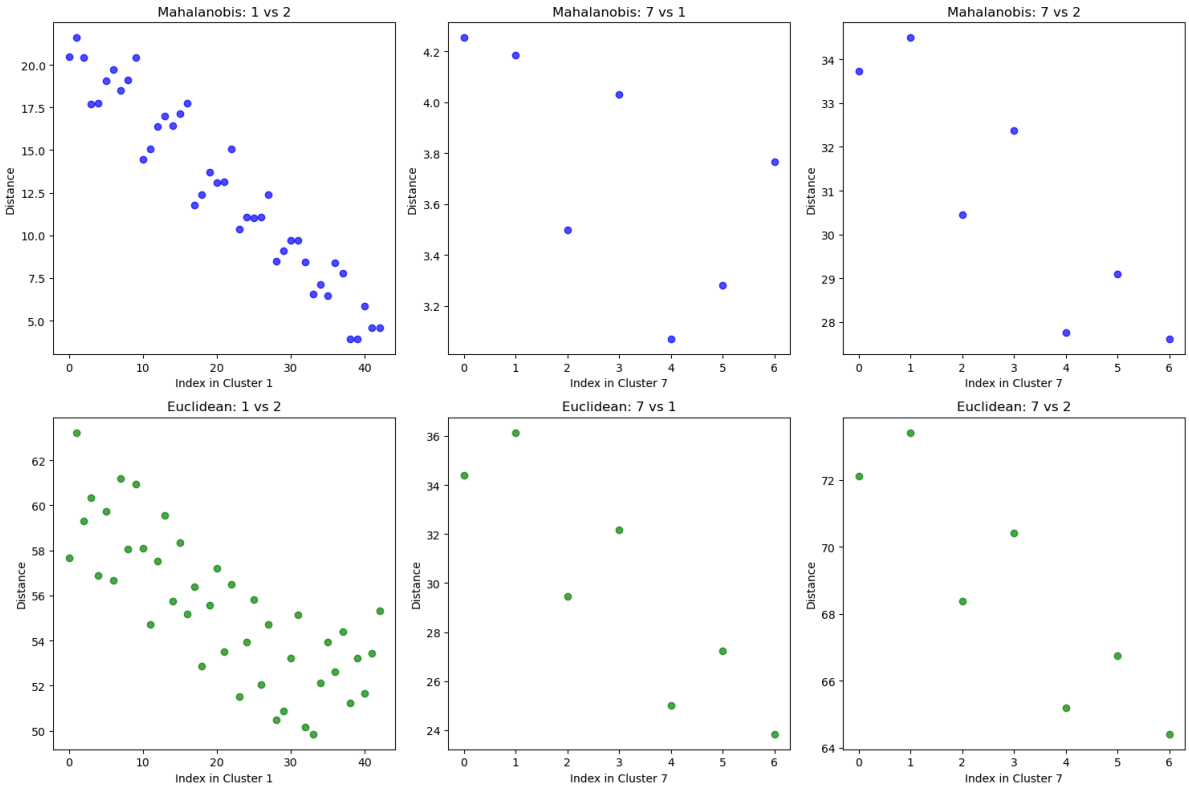
Look at cluster pairs 7,1 and 7,2. Although 7 is twice as distant from 2 as it is from 1, its mahalanobis distance is almost ten times as distant from 2 as it is from 1.

This is because in dimension X, cluster 2 has large variance, so deviations from mean in X contribute to a lower mahalanobis distance and in dimension Y, has lower variance so deviations from mean in Y contribute to higher mahalanobis distance. This is the net effect of normalizing the euclidean distance by the covariance matrix. Next I use the p value based on this distance to calculate the likelihood of points in cluster j belonging to cluster i. I use a threshold value (p_{ij}) of 0.1

In [23]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2

# Load the data from the .npy file
data = filtered_data

# Extract columns for clusters, x, y, z
clusters = data[:, 0]   # cluster labels
xyz_data = data[:, 1:4]  # [x, y, z] coordinates

# Define the pairs of clusters to analyze
pairs = [(1, 2), (7, 1), (7, 2)]

# Prepare the figure for the plots with a 2x3 grid (for 3 pairs)
plt.figure(figsize=(15, 10))

for idx, (cluster_a, cluster_b) in enumerate(pairs, start=1):
    # Get the data points for the clusters
    cluster_a_data = xyz_data[clusters == cluster_a]
```

```
    cluster_b_data = xyz_data[clusters == cluster_b]

    # Check if cluster_b_data has enough unique points
    if cluster_b_data.shape[0] < 2:
        print(f'Skipping pair ({cluster_a}, {cluster_b}) due to insufficient data i
        continue

    # Calculate the mean vector for the reference cluster (cluster B)
    mean_b = np.mean(cluster_b_data, axis=0)

    # Calculate the covariance matrix for cluster B
    cov_matrix = np.cov(cluster_b_data.T)
    inv_cov_matrix = np.linalg.inv(cov_matrix)

    # Calculate the Mahalanobis distances for points in cluster A against the mean
    mahalanobis_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        mahalanobis_distance = np.sqrt(diff @ inv_cov_matrix @ diff.T)
        mahalanobis_distances.append(mahalanobis_distance)

    # Calculate the Euclidean distances for points in cluster A against the mean of
    euclidean_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        euclidean_distance = np.sqrt(diff @ diff.T)
        euclidean_distances.append(euclidean_distance)


    # Print the averages and p-values
    avg_mahalanobis = np.mean(mahalanobis_distances)
    avg_euclidean = np.mean(euclidean_distances)
    # Calculate and print the p-values for Mahalanobis distances
    dof = 2  # Degrees of freedom (3 dimensions: x, y, z)
    p_value = 1 - chi2.cdf(avg_mahalanobis, df=dof)  # Survival function (1 - CDF)
    avg_p_value = p_value

    print(f'Pair ({cluster_a}, {cluster_b}):')
    print(f'  Average Mahalanobis distance: {avg_mahalanobis:.3f}')
    print(f'  Average Euclidean distance: {avg_euclidean:.3f}')
    print(f'  Chi Square: {chi2.cdf(avg_mahalanobis, df=dof) :.3f}')
    print(f'  p-value (Mahalanobis): {avg_p_value:.3f}')

    # Plotting Mahalanobis Distance
    plt.subplot(2, 3, idx)
    plt.scatter(range(len(mahalanobis_distances)), mahalanobis_distances, color='bl
    plt.title(f'Mahalanobis: {cluster_a} vs {cluster_b}')
    plt.xlabel(f'Index in Cluster {cluster_a}')
    plt.ylabel('Distance')

    # Plotting Euclidean Distance
    plt.subplot(2, 3, idx + 3)
    plt.scatter(range(len(euclidean_distances)), euclidean_distances, color='green'
    plt.title(f'Euclidean: {cluster_a} vs {cluster_b}')
    plt.xlabel(f'Index in Cluster {cluster_a}')
    plt.ylabel('Distance')

# Show the plot
plt.tight_layout()
plt.show()
```

```
Pair (1, 2):
  Average Mahalanobis distance: 12.758
  Average Euclidean distance: 55.369
  Chi Square: 0.998
  p-value (Mahalanobis): 0.002
Pair (7, 1):
  Average Mahalanobis distance: 3.727
  Average Euclidean distance: 29.745
  Chi Square: 0.845
  p-value (Mahalanobis): 0.155
Pair (7, 2):
  Average Mahalanobis distance: 30.790
  Average Euclidean distance: 68.671
  Chi Square: 1.000
  p-value (Mahalanobis): 0.000
```



Continue reading

Pair (1, 2):

Average Mahalanobis distance: 12.758

Average Euclidean distance: 55.369

Chi Square: 0.998

p-value (Mahalanobis): 0.002

Pair (7, 1):

Average Mahalanobis distance: 3.727

Average Euclidean distance: 29.745

Chi Square: 0.845

p-value (Mahalanobis): 0.155

Pair (7, 2):

Average Mahalanobis distance: 30.790

Average Euclidean distance: 68.671

Chi Square: 1.000

p-value (Mahalanobis): 0.000

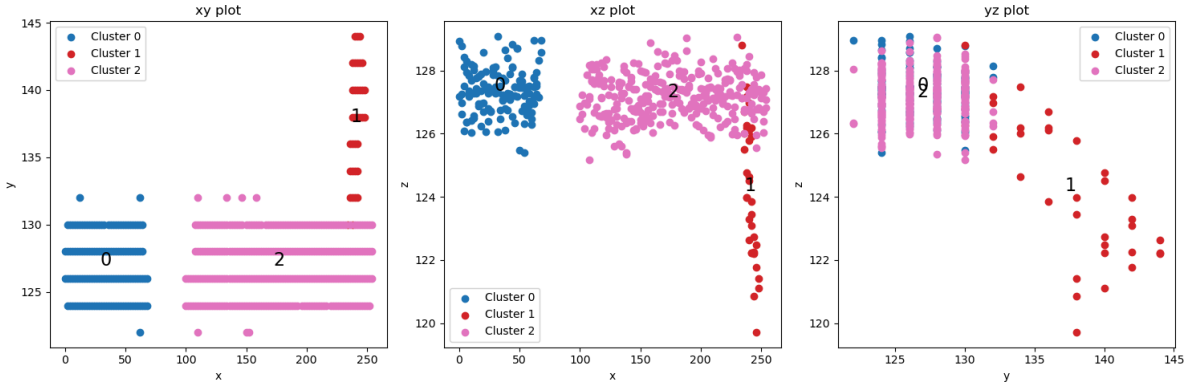
Note that points in cluster 7 has more than 15% chance of belonging to Cluster 1 (around 85% points in cluster 1 has mahalanobis distance less than 3.7)

Now lets look into some more events

For example at 0MeV and 1 degree in the center of mass.short track at high angle

In [29]:
```
import numpy as np
import matplotlib.pyplot as plt

data = np.load('event_8.npy')

# Extract columns
cluster_ids = data[:, 0].astype(int)
x = data[:, 1]
y = data[:, 2]
z = data[:, 3]

# Get unique clusters
unique_clusters = np.unique(cluster_ids)

# Set up the plot
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Create a colormap for clusters (use the colormap name without the length argument
colors = plt.colormaps.get_cmap('tab10')

# Scatter plots for each subplot (xy, xz, yz)
for idx, cluster in enumerate(unique_clusters):
    # Mask for the current cluster
    cluster_mask = cluster_ids == cluster

    # Get points belonging to the current cluster
    x_cluster = x[cluster_mask]
    y_cluster = y[cluster_mask]
    z_cluster = z[cluster_mask]

    # Plot xy
    axes[0].scatter(x_cluster, y_cluster, color=colors(idx / len(unique_clusters)),

    # Plot xz
    axes[1].scatter(x_cluster, z_cluster, color=colors(idx / len(unique_clusters)),

    # Plot yz
    axes[2].scatter(y_cluster, z_cluster, color=colors(idx / len(unique_clusters)),

    # Calculate cluster means
    mean_x = np.mean(x_cluster)
    mean_y = np.mean(y_cluster)
    mean_z = np.mean(z_cluster)

    # Plot cluster ID near the mean points
    axes[0].text(mean_x, mean_y, f'{cluster}', fontsize=16, ha='center')
    axes[1].text(mean_x, mean_z, f'{cluster}', fontsize=16, ha='center')
    axes[2].text(mean_y, mean_z, f'{cluster}', fontsize=16, ha='center')

# Set titles and labels
axes[0].set_title('xy plot')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')

axes[1].set_title('xz plot')
```

```
axes[1].set_xlabel('x')
axes[1].set_ylabel('z')

axes[2].set_title('yz plot')
axes[2].set_xlabel('y')
axes[2].set_ylabel('z')

# Add legends
for ax in axes:
    ax.legend()

# Display the plots
plt.tight_layout()
plt.show()
```

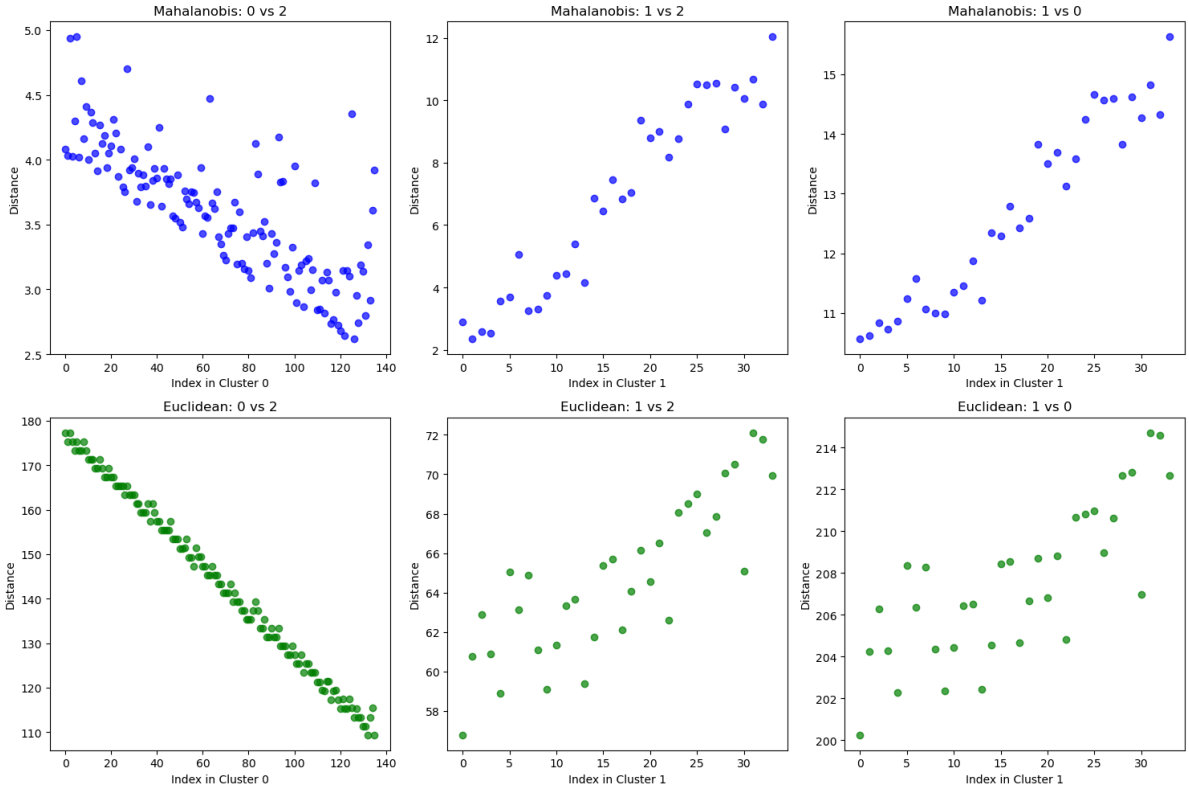


In [28]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2

# Load the data from the .npy file
data = np.load('event_8.npy')

# Extract columns for clusters, x, y, z
clusters = data[:, 0]   # cluster labels
xyz_data = data[:, 1:4]  # [x, y, z] coordinates

# Define the pairs of clusters to analyze
pairs = [(0, 2), (1, 2), (1, 0)]

# Prepare the figure for the plots with a 2x3 grid (for 3 pairs)
plt.figure(figsize=(15, 10))

for idx, (cluster_a, cluster_b) in enumerate(pairs, start=1):
    # Get the data points for the clusters
    cluster_a_data = xyz_data[clusters == cluster_a]
    cluster_b_data = xyz_data[clusters == cluster_b]

    # Check if cluster_b_data has enough unique points
    if cluster_b_data.shape[0] < 2:
        print(f'Skipping pair ({cluster_a}, {cluster_b}) due to insufficient data i
        continue

    # Calculate the mean vector for the reference cluster (cluster B)
    mean_b = np.mean(cluster_b_data, axis=0)

    # Calculate the covariance matrix for cluster B
    cov_matrix = np.cov(cluster_b_data.T)
    inv_cov_matrix = np.linalg.inv(cov_matrix)

    # Calculate the Mahalanobis distances for points in cluster A against the mean
    mahalanobis_distances = []
```

```
    for point in cluster_a_data:
        diff = point - mean_b
        mahalanobis_distance = np.sqrt(diff @ inv_cov_matrix @ diff.T)
        mahalanobis_distances.append(mahalanobis_distance)

    # Calculate the Euclidean distances for points in cluster A against the mean of
    euclidean_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        euclidean_distance = np.sqrt(diff @ diff.T)
        euclidean_distances.append(euclidean_distance)


    # Print the averages and p-values
    avg_mahalanobis = np.mean(mahalanobis_distances)
    avg_euclidean = np.mean(euclidean_distances)
    # Calculate and print the p-values for Mahalanobis distances
    dof = 2  # Degrees of freedom (3 dimensions: x, y, z)
    p_value = 1 - chi2.cdf(avg_mahalanobis, df=dof)  # Survival function (1 - CDF)
    avg_p_value = p_value

    print(f'Pair ({cluster_a}, {cluster_b}):')
    print(f'  Average Mahalanobis distance: {avg_mahalanobis:.3f}')
    print(f'  Average Euclidean distance: {avg_euclidean:.3f}')
    print(f'  Chi Square: {chi2.cdf(avg_mahalanobis, df=dof) :.3f}')
    print(f'  p-value (Mahalanobis): {avg_p_value:.3f}')

    # Plotting Mahalanobis Distance
    plt.subplot(2, 3, idx)
    plt.scatter(range(len(mahalanobis_distances)), mahalanobis_distances, color='bl
    plt.title(f'Mahalanobis: {cluster_a} vs {cluster_b}')
    plt.xlabel(f'Index in Cluster {cluster_a}')
    plt.ylabel('Distance')

    # Plotting Euclidean Distance
    plt.subplot(2, 3, idx + 3)
    plt.scatter(range(len(euclidean_distances)), euclidean_distances, color='green'
    plt.title(f'Euclidean: {cluster_a} vs {cluster_b}')
    plt.xlabel(f'Index in Cluster {cluster_a}')
    plt.ylabel('Distance')

# Show the plot
plt.tight_layout()
plt.show()
```

```
Pair (0, 2):
  Average Mahalanobis distance: 3.599
  Average Euclidean distance: 143.577
  Chi Square: 0.835
  p-value (Mahalanobis): 0.165
Pair (1, 2):
  Average Mahalanobis distance: 6.874
  Average Euclidean distance: 64.700
  Chi Square: 0.968
  p-value (Mahalanobis): 0.032
Pair (1, 0):
  Average Mahalanobis distance: 12.680
  Average Euclidean distance: 207.508
  Chi Square: 0.998
  p-value (Mahalanobis): 0.002
```



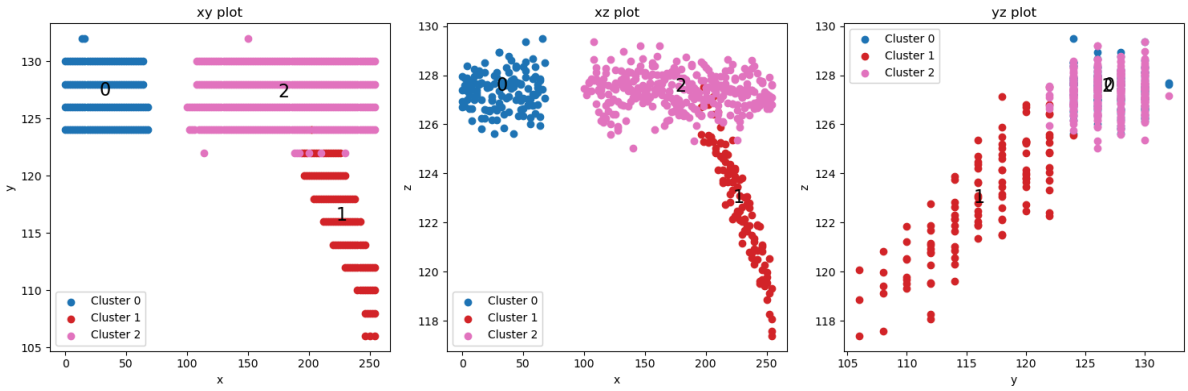
Continue reading

Notice how, points in cluster 0 has more than 16 % chance of belonging to cluster 2, while points in cluster 1 (the scattered track) has only around 3% chance of belonging to the cluster 2 (the beam track)

Now lets look at another event 25MeV and 1CM, low angled track

In [30]:
```
import numpy as np
import matplotlib.pyplot as plt

data = np.load('event_9.npy')

# Extract columns
cluster_ids = data[:, 0].astype(int)
x = data[:, 1]
y = data[:, 2]
z = data[:, 3]

# Get unique clusters
unique_clusters = np.unique(cluster_ids)

# Set up the plot
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Create a colormap for clusters (use the colormap name without the length argument
colors = plt.colormaps.get_cmap('tab10')

# Scatter plots for each subplot (xy, xz, yz)
for idx, cluster in enumerate(unique_clusters):
    # Mask for the current cluster
    cluster_mask = cluster_ids == cluster

    # Get points belonging to the current cluster
    x_cluster = x[cluster_mask]
    y_cluster = y[cluster_mask]
    z_cluster = z[cluster_mask]
```

```
    # Plot xy
    axes[0].scatter(x_cluster, y_cluster, color=colors(idx / len(unique_clusters)),

    # Plot xz
    axes[1].scatter(x_cluster, z_cluster, color=colors(idx / len(unique_clusters)),

    # Plot yz
    axes[2].scatter(y_cluster, z_cluster, color=colors(idx / len(unique_clusters)),

    # Calculate cluster means
    mean_x = np.mean(x_cluster)
    mean_y = np.mean(y_cluster)
    mean_z = np.mean(z_cluster)

    # Plot cluster ID near the mean points
    axes[0].text(mean_x, mean_y, f'{cluster}', fontsize=16, ha='center')
    axes[1].text(mean_x, mean_z, f'{cluster}', fontsize=16, ha='center')
    axes[2].text(mean_y, mean_z, f'{cluster}', fontsize=16, ha='center')

# Set titles and labels
axes[0].set_title('xy plot')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')

axes[1].set_title('xz plot')
axes[1].set_xlabel('x')
axes[1].set_ylabel('z')

axes[2].set_title('yz plot')
axes[2].set_xlabel('y')
axes[2].set_ylabel('z')

# Add legends
for ax in axes:
    ax.legend()

# Display the plots
plt.tight_layout()
plt.show()
```



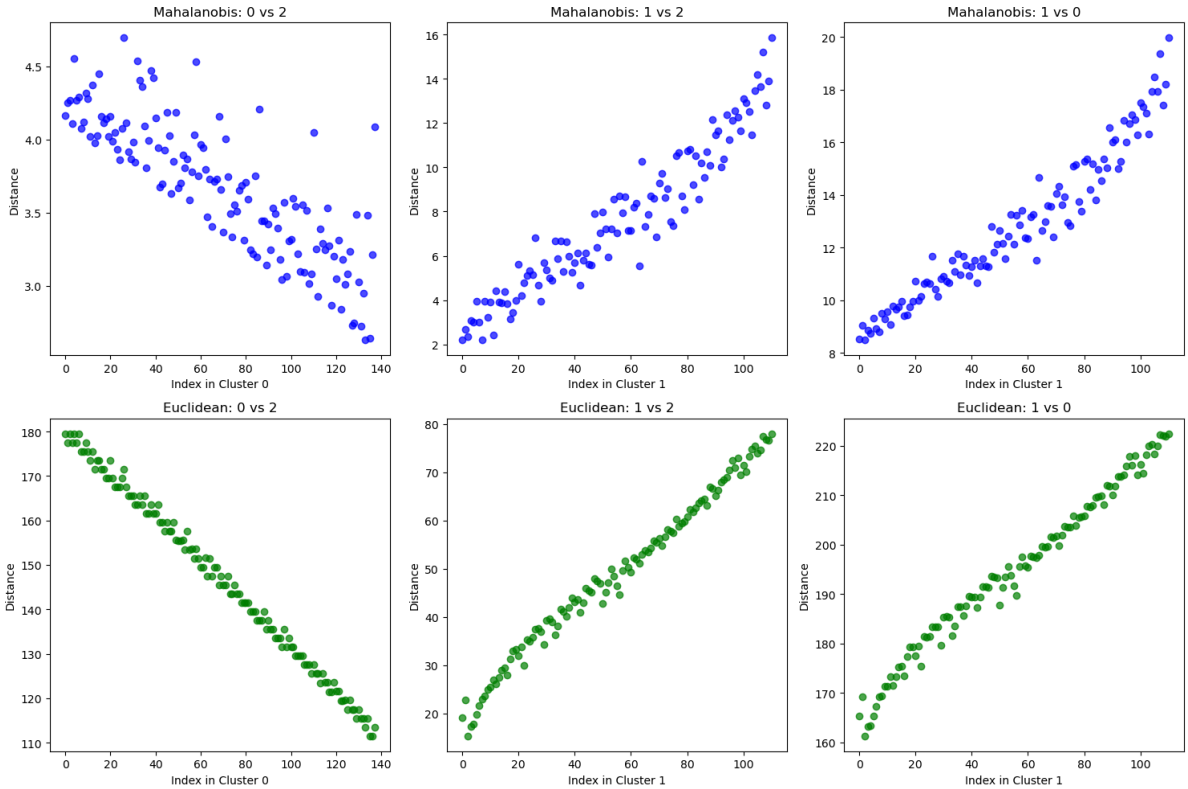
In [31]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2

# Load the data from the .npy file
data = np.load('event_9.npy')

# Extract columns for clusters, x, y, z
clusters = data[:, 0]   # cluster labels
xyz_data = data[:, 1:4]  # [x, y, z] coordinates
```

```
# Define the pairs of clusters to analyze
pairs = [(0, 2), (1, 2), (1, 0)]

# Prepare the figure for the plots with a 2x3 grid (for 3 pairs)
plt.figure(figsize=(15, 10))

for idx, (cluster_a, cluster_b) in enumerate(pairs, start=1):
    # Get the data points for the clusters
    cluster_a_data = xyz_data[clusters == cluster_a]
    cluster_b_data = xyz_data[clusters == cluster_b]

    # Check if cluster_b_data has enough unique points
    if cluster_b_data.shape[0] < 2:
        print(f'Skipping pair ({cluster_a}, {cluster_b}) due to insufficient data i
        continue

    # Calculate the mean vector for the reference cluster (cluster B)
    mean_b = np.mean(cluster_b_data, axis=0)

    # Calculate the covariance matrix for cluster B
    cov_matrix = np.cov(cluster_b_data.T)
    inv_cov_matrix = np.linalg.inv(cov_matrix)

    # Calculate the Mahalanobis distances for points in cluster A against the mean
    mahalanobis_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        mahalanobis_distance = np.sqrt(diff @ inv_cov_matrix @ diff.T)
        mahalanobis_distances.append(mahalanobis_distance)

    # Calculate the Euclidean distances for points in cluster A against the mean of
    euclidean_distances = []
    for point in cluster_a_data:
        diff = point - mean_b
        euclidean_distance = np.sqrt(diff @ diff.T)
        euclidean_distances.append(euclidean_distance)


    # Print the averages and p-values
    avg_mahalanobis = np.mean(mahalanobis_distances)
    avg_euclidean = np.mean(euclidean_distances)
    # Calculate and print the p-values for Mahalanobis distances
    dof = 2  # Degrees of freedom (3 dimensions: x, y, z)
    p_value = 1 - chi2.cdf(avg_mahalanobis, df=dof)  # Survival function (1 - CDF)
    avg_p_value = p_value

    print(f'Pair ({cluster_a}, {cluster_b}):')
    print(f'  Average Mahalanobis distance: {avg_mahalanobis:.3f}')
    print(f'  Average Euclidean distance: {avg_euclidean:.3f}')
    print(f'  Chi Square: {chi2.cdf(avg_mahalanobis, df=dof) :.3f}')
    print(f'  p-value (Mahalanobis): {avg_p_value:.3f}')

    # Plotting Mahalanobis Distance
    plt.subplot(2, 3, idx)
    plt.scatter(range(len(mahalanobis_distances)), mahalanobis_distances, color='bl
    plt.title(f'Mahalanobis: {cluster_a} vs {cluster_b}')
    plt.xlabel(f'Index in Cluster {cluster_a}')
    plt.ylabel('Distance')

    # Plotting Euclidean Distance
    plt.subplot(2, 3, idx + 3)
    plt.scatter(range(len(euclidean_distances)), euclidean_distances, color='green'
    plt.title(f'Euclidean: {cluster_a} vs {cluster_b}')
```

```
    plt.xlabel(f'Index in Cluster {cluster_a}')
    plt.ylabel('Distance')

# Show the plot
plt.tight_layout()
plt.show()
```

```
Pair (0, 2):
  Average Mahalanobis distance: 3.676
  Average Euclidean distance: 146.747
  Chi Square: 0.841
  p-value (Mahalanobis): 0.159
Pair (1, 2):
  Average Mahalanobis distance: 7.653
  Average Euclidean distance: 48.944
  Chi Square: 0.978
  p-value (Mahalanobis): 0.022
Pair (1, 0):
  Average Mahalanobis distance: 12.834
  Average Euclidean distance: 194.574
  Chi Square: 0.998
  p-value (Mahalanobis): 0.002
```



Continue reading

Notice how, points in cluster 0 has more than 15 % chance of belonging to cluster 2, while points in cluster 1 (the scattered track) has only around 2% chance of belonging to the cluster 2 (the beam track)

In [ ]: